

Duvarlarla İlerlemek: Test GÜdÜmlü Geliştirmeden Ölçüt Önceliği İlkesine

Ömer Faruk Yavuz

Temmuz 2026

Giriş ve Sorunun Konumlandırılması

Bu metin, test güdümlü geliştirmenin (TDD) ne tür bir yöntem olduğu sorusundan başlayıp, bu yöntemin altında yatan daha genel bir bilişsel ve kurumsal kalıbın çıkarımına ulaşan bir tartışmanın kayıdıdır. Başlangıç sorusu ikili bir seçenek biçiminde kurulmuştur: TDD yalnızca bir sıralama kuralı mıdır (önce test, sonra kod), yoksa bilimsel hipotez testinin mühendislik alanına aktarılmış bir versiyonu mudur? Tartışmanın vardığı sonuç, her iki şıkkın da eksik olduğu ve doğru tanımın üçüncü bir yerde durduğudur.

TDD’nin Doğası

“Önce Test Yaz” Okumasının Yetersizliği

Bu okuma, ritüeli mekanizmayla karıştırır. Kent Beck’in özgün çerçevelemesi bir doğruluk kanıtı yöntemi değil, bir korku yönetimi aracıdır: adım boyu küçültüldüğünde, hatanın işlendiği an ile fark edildiği an arasındaki mesafe kısalmır. Bu, epistemolojik değil psikolojik bir argümandır.

Bununla birlikte yöntem salt ritüel de değildir; ritüelin bir bileşeni kasıtlı olarak epistemiktir. Kırmızı fazın zorunluluğu, yani testin önce başarısız olduğunun gözlemlenmesi şartı, deney tasarımındaki negatif kontrolün doğrudan karşılığıdır. İlk yazımında geçen bir test, hiçbir şey ölçmediği fark edilmemiş bir alettir; sapmasızlığı gösterilmemiş bir ölçüm aygıtıyla yapılan ölçüme benzer.

“Hipotez Testi” Okumasının Yetersizliği

Bu okuma temel bir asimetriyi gözden geçirir. Bilimsel hipotez testinde dünya sabit, hipotez esnektir; çelişki durumunda hipotez terk edilir. TDD’de ise ilişki terstir: test kırmızı yandığında testi değil, kodu değiştiririz.

```
Bilim: hipotez <-- revizyon <-- [çelişki] --> dünya sabit
TDD:   test sabit --> [çelişki] --> revizyon --> kod
```

Dolayısıyla test, hipotez rolünde değil, aksiyom veya şartname rolündedir. Bu haliyle TDD, deneyden çok yapıcı (constructive) ispata benzer: önce ne kanıtlanacağı beyan edilir, ardından kanıt inşa edilir.

Terimin istatistiksel anlamı da uymaz. Örnek tabanlı bir testte örneklem, dağılım ve p -değeri yoktur; deterministik ve tek noktalı bir değerlendirme vardır. Popperci yanlışlanabilirlik ile Fisher/Neyman–Pearson çıkarımı burada ayrılmalıdır; TDD’nin uzak akrabası varsa birincisidir, o da gevşek anlamda.

Lakatos’çu Okumanın Üstünlüğü

Proofs and Refutations’taki tablo, TDD’ye Popper’inkinden daha iyi oturur: ne teorem ne tanım sabittir; karşı örnek baskısı altında ikisi birlikte evrilir. Gerçek TDD döngüsünde de test ve kod eşzamanlı olgunlaşır. Kırmızı durumda kimi zaman kod düzeltilir, kimi zaman “bu davranışı yanlış tarif etmişim” denerek test düzeltilir. Sabit taraf yoktur; çoğu zaman yapılan şey canavar dışlamadır (monster-barring).

Analojinin Literal Olarak Geçerli Olduğu Modlar

Hipotez testi benzetmesi üç bağlamda mecazi olmaktan çıkıp düz anlamıyla geçerlidir.

- **Karakterizasyon testleri.** Miras kodda sistemin ne yaptığı bilinmez. “Sanırım şunu döndürüyor” biçiminde bir test yazılır ve sistem tarafından yalanlanabilir. Burada kod sabit dünya, test ise gerçekten yanlışlanabilir bir sanıdır.
- **Hata ayıklama.** İzole edici deney ve ayırıcı tanı yapısı; test daraltılarak nedensel iddia sınanır.
- **Özellik tabanlı test.** Evrensel bir iddia beyan edilir, makine karşı örnek arar. Popperci şemayı örnek tabanlı TDD’den çok daha harfi harfine uygular.

İşleyen Mekanizmalar

Bağımsız İkinci Türetme

TDD’nin epistemik değerini en iyi açıklayan analogi çift taraflı muhasebedir. Test ve kod, aynı niyetin iki bağımsız türetmesidir; uyuşmaları kanıt değeri taşır çünkü bağımsızdırlar.

Bu, en yaygın patolojiyi de açıklar. Test implementasyonun yapısından türetildiğinde (her iş birliği nesnesinin taklit edilmesi, çağrı sıralarının doğrulanması), bağımsızlık çöker ve geriye totoloji kalır: kodun kod olduğunu doğrulayan bir test. Bu, yanlışlanamaz hipotezin yazılımdaki karşılığıdır ve yoğun taklit içeren sütlerin yemyeşil yanarken sistemin çalışmamasının sebebidir.

Geri Besleme Gecikmesinin Kısaltılması

Değerli olan tek bir testin doğruluğu değil, yanılmanın öğrenilme gecikmesinin kısalığıdır. Bu açıdan TDD bir epistemoloji değil, bir kontrol döngüsü tasarımıdır. Sütün çeşitliliğinin sistemin arıza modu çeşitliliğine denk gelmesi gerekliliği, Ashby'nin gerekli çeşitlilik yasasının gevşek bir analojisidir.

Test Yazma Zorluğunun Ölçüm Aleti Olması

Testi kurmak için çok sayıda bağımlılığın ayağa kaldırılması gerekiyorsa, bu tasarım hakkında ampirik bir veridir. Burada hipotez testi yapılmamakta, termometre okunmaktadır.

Yanışlama Menzili

TDD'nin yürütebildiği iddia sınıfı sanıldığından dardır: birim düzeyinde davranış. Entegrasyon davranışı, zamanlama, yük altındaki davranış ve “doğru şeyi mi yapıyorum” sorusu menzilin dışındadır. Yanışlama gücüne göre kaba bir sıralama:

1. Örnek tabanlı test: tek doğrulayıcı örnek
2. Özellik tabanlı test: evrensel iddia ve düşmanca karşı örnek arama
3. Tip sistemi: bir özellik sınıfının çalıştırmasız ispatı
4. Formal doğrulama: ispat

Toparlayıcı tanım şudur: TDD, yanışlamanın *biçimini* ödünç alan bir tasarım disiplini; işleyen mekanizması ise geri besleme gecikmesinin kısaltılması ve bağımsız ikinci türetmedir.

Kırılğanlık ve Kaskad Sorunu

Kenetlenme Yüzeyi

Tek bir değişikliğin çok sayıda testi düşürmesi olgusu, TDD'nin değil testlerin neye kenetlendiğinin sonucudur.

davranışa kenetli test -> içeri değişir, test yeşil kalır
yapıya kenetli test -> içeri değişir, N test kırmızı
(taklitler, çağrı sırası, iç durum doğrulamaları)

Kırk test aynı iç ayrıntıya dokunuyorsa, o ayrıntı kırk kenarlı bir düğüme dönüşmüştür. Literatürdeki adı kırılğan test problemi veya aşırı belirtilmiş yazılımdır (Meszaros). Test süiti, API'nin ikinci tüketicisidir; her test bir bağımlılık kenarıdır.

Kaskadın ikinci yaygın kaynağı paylaşılan kurulum bloklarıdır. Ortak kurulum, süitin gizli tekil noktasıdır; oradaki bir varsayım değiştiğinde bağımsız görünen testler topluca düşer.

TDD'nin Takası

Kırmızı–yeşil–düzenle döngüsü bir eksen ucuzlatırken diğerini pahalılaştırır:

- İç yapı değişimi: ucuz (yeşil altında yeniden düzenleme)
- Arayüz veya kavram değişimi: pahalı, ve maliyet test sayısı ile çarpılır

Beck'in test desiderata çerçevesi bu gerilimi açıkça isimlendirir: test hem hassas hem yapıya duyarsız olmak ister, bu iki nitelik birbirini çeker. Aynı yerde testlerin silinebilir olduğu da belirtilir; süit kutsal değil, bir maliyet–fayda kalemidir.

Yıkım Ölçeğinin Bilimle Karşılaştırılması

“TDD’de yıkım hipotez testine göre daha büyüktür” sezgisi, görünürlük ile ölçeği karıştırmaktan doğar. Duhem–Quine tezine göre bilimde hiçbir hipotez tek başına sınanmaz; hipotez, yardımcı varsayımlar ve ölçüm aletinin teorisi birlikte sınanır. Lakatos’un koruyucu kuşağı ile Kuhn’un paradigma değişimi, yıkımın yazılımdakinden çok daha geniş olabildiğini gösterir.

	TDD	Bilim
Görünürlük	anında, mekanik	yavaş, dağınık
Lokalizasyon	tam	Duhem–Quine gereği belirsiz
Birim maliyet	ucuz, mekanik onarım	program ölümü

TDD’nin yıkımı gürültülü fakat ucuz, bilimin yıkımı sessiz fakat pahalıdır. TDD maliyeti öne çeker ve görünür kılar.

Yıkım Büyüklüğünün Tanısal Kullanımı

Tek bir tasarım kararı değiştiğinde çok sayıda test kırmızı yanıyorsa:

- Dışa dönük davranış değişmediyse, kusur testlerdedir; süit yapıya kenetlenmiştir.
- Davranış gerçekten değiştiyse, kırmızılar doğru ve bilgilendiricidir.

Faz Hatası

Kaskadın en sık nedeni, tasarımın şekli henüz bilinmezken test yazmaktır. TDD’nin maliyet eğrisi, “ne” sorusunun kabaca yanıtlanmış varsayar; keşif fazında değil, iniş fazında verimlidir. Beck’in önerdiği yol *spike and stabilize* biçimindedir: keşif kodu yazılır, çalıştırılır, öğrenilir, atılır; ardından öğrenilen şekle TDD başlatılır.

Sonradan Test ile TDD Arasındaki Fark

Görünen fark sıralamadır:

```
Sonradan test: kod -> test -> çalıştır
TDD:          test -> kırmızı -> kod -> yeşil -> düzenle
```

Asıl farklar ise şunlardır:

- **İşlev.** Sonradan yazılan test bir denetim aracıdır; TDD’de test bir tasarım aracıdır. Testi yazarken cevaplanan soru “bu birimi nasıl çağırmak isterim” sorusudur; API, ilk tüketicisi üzerinden tasarlanır.
- **Kırmızı zorunluluğu.** Sonradan yazılan testte aletin doğrulanması adımı yoktur; test yanlışlıkla daima yeşil olabilir.
- **Bağımsızlık.** Kod yazıldıktan sonra yazılan test, kodun şekline göre biçimlenir ve onu tekrar anlatır.
- **Gecikme.** Geri bildirim ölçeği günlerden dakikalara iner.
- **Tasarım sinyali.** Test kurulumunun zorlaşması, kod yazılmadan önce elde edilen bir tasarım verisidir.

Kırılganlık meselesi bu iki yaklaşımın farkı değil, ortak sorunudur; TDD test sayısını artırdığı için sorunu daha görünür kılar.

TDD’nin Değer Sıralaması

Yaygın kanının aksine, faydaların önem sırası şöyledir:

1. Kodu şekillendirmesi (test edilebilirlik ile iyi tasarımın büyük ölçüde örtüşmesinden yararlanır)
2. Değişikliği güvenli ve dolayısıyla mümkün kılması
3. Varsayımları açığa çıkarması
4. Hatayı erken göstermesi

Dördüncü madde en somut olduğu için en çok konuşulandır; oysa yalnızca onun için TDD uygulayan biri, kodu yazdıktan sonra test yazarak benzer faydayı elde eder.

Kalıbın Soyutlanması

“Test” ve “kod” terimleri çıkarıldığında geriye şu iskelet kalır:

1. Hedefi, ulaşıp ulaşılmadığı makinece denetlenebilecek biçimde önceden yaz
2. Şu an ulaşılmadığını doğrula
3. En küçük adımla ulaş
4. Hedef bozulmadan içeriği düzenle
5. Tekrarla

Tek cümlede: **başarı ölçütünü üretimden önce ve dışarıdan tanımla, sonra aradaki farkı kapat.** Kritik nitelik “dışarıdan”dır; ölçüt üretim başladıktan sonra yazılırsa üretilen şeye göre biçimlenir ve kendini onaylar.

Kalıbın İşleme Mekanizmaları

- Ölçüt kaymasının engellenmesi
- Ölçüm aletinin doğrulanması
- İki bağımsız türetmenin karşılaştırılması

Geçerlilik Koşulları

- Hedef ölçülebilir olmalıdır
- Hedef önceden gerçekten bilinmelidir; keşif fazında ölçüt erken donar
- Adım küçük ve geri alınabilir olmalıdır
- Ölçüt dışa bakmalıdır; içe bakan ölçüt düzeltmeyi imkânsızlaştırır

Kalıbın Karanlık Tarafı

Ölçülebilir hedef koymamanın bedeli, ölçülemeyen her şeyin kaybolmasıdır. Ölçüt, iyileştirilmek istenen şeyin yerini almaya başlar; Goodhart yasasının genel biçimi budur.

Kalıbın Diğer Görünümleri

Sonradan Kendini Kandırmayı Engelleme

Ön kayıt ve kayıtlı raporlar (HARKing’in engellenmesi), tutulan test kümesi ve veri sızıntısının önlenmesi, anlamlılık eşiğinin deneyden önce sabitlenmesi, kör değerlendirme ve kör hakemlik, yatırım tezinin çıkış ölçütüyle birlikte önceden yazılması.

Gelecekteki Zayıflığa Karşı Bağlanma

Odysseus'un direğe bağlanması, vasiyet ve önceden hasta talimatı, anayasa ve usul kuralları, Rawls'un cehalet perdesi, "biri keser diğeri seçer" prosedürü, Gollwitzer'in uygulama niyetleri, bütçe ve zarar durdurma emirleri.

Ölçüm Aletinin Doğrulanması

Negatif kontrol, kalibrasyon, geçer/geçmez mastarı, mutasyon testi.

Ölçütle Yetki Devri

Auftragstaktik ve komutan niyeti, kabul kriterleri ve fabrika kabul testi, sözleşmeyle tasarım (Meyer), poka-yoke.

Formun Üretimden Önce Geldiği Alanlar

Kata, kebiki çizgisi, kalıp ve döküm, Fux'un tür kontrpuanı, on iki ölçülük blues ve sonat formu, storyboard ve animatik, geriye dönük tasarım (Wiggins ve McTighe), Dogme 95 yemini.

Kalıbın Bozulduğu Yerler

Durum	Sonuç
Smava göre öğretim	Ölçüt hedefi yer değiştirir
KPI yönetimi	Ölçülmeyen görünmez olur
Şelale modeli	Alan bilinmezken şartname donar
Keşif araştırmasında ön kayıt	Beklenmedik bulgu meşrulaştırılmaz
Merkezi plan hedefleri	Ölçütün harfine uyan çarpık üretim
İçe bakan kabul kriteri	Değiştirilemezlik

Uygarlık Ölçeğindeki Kazanım

Bu kalıbın toplam etkisi, güvenin kişiden prosedüre taşınmasıdır. Uygarlığın ölçek kazanması bunun sonucudur.

Kazanımlar

- **Yabancıyla işbirliği.** Sözleşme, standart ağırlık ve ölçüler, çift taraflı muhasebe ve akreditif, karşı tarafı tanıma zorunluluğunu ortadan kaldırmıştır.
- **Değiştirilebilir parça.** Tolerans kavramı, yani parça üretilmeden önce yazılmış geçer/geçmez ölçüsü, birbirini görmemiş üreticilerin parçalarının uyumunu mümkün kılmıştır.
- **Bilginin birikmesi.** Yöntemin önceden ve açıkça yazılması, sonucun yinelenebilirliğini sağlamıştır.

- **Kurumların kurucularını aşması.** Uyuşmazlıktan önce yazılmış kural, kurumu kişiden bağımsızlaştırır.
- **Geri dönülebilirlik.** Ölçüt ve yedek varsa riskli değişiklik denenebilir; yoksa donma oluşur.
- **Hatanın bilgiye dönüşmesi.** Ölçüt önceden ve dışarıda yazılıysa yanılğı kişisel kusur olmaktan çıkıp veri hâline gelir.
- **Cırcır etkisi.** Yazılı ölçüt geri kaymayı engelleyen dıřtır; yazılmadığında bilgi örtük kalır ve taşıyıcısıyla birlikte kaybolur.
- **Vasatın üretken kılınması.** Dehanın ölçeđi yoktur; prosedür, ortalama bir icracının güvenilir çıktı üretmesini mümkün kılar.

ölçütsüz: çıktı kalitesi ~ kişinin kalitesi -> ölçeklenmez
ölçütlü: çıktı kalitesi ~ prosedürün kalitesi -> ölçeklenir

Bedeller

Ölçülemeyenin yok sayılması (Scott'ın *Seeing Like a State*'inde bilimsel ormancılık örneđi), zanaat bilgisinin ve aktarılamayan yargının değersizleşmesi, Goodhart yasasının uygarlık ölçeğinde işlemesi, ölçütün kendisinin amaca dönüşmesi olarak bürokrasi.

Sonuç

Duvarlarla ilerlemek, uygarlığın erdemi gereksizleştirme stratejisidir. Dürüst, yetenekli, hafızası sağlam ve niyeti iyi bireylere bağımlı kalmak yerine, bunların hiçbirini varsaymayan yapılar kurulmuştur. Kazanılan ölçek ve süreklilik, kaybedilen ise ölçüte sığmayan her şeydir. Sonuç ikili bir tercih deđil bir ayar meselesidir: ölçüt ne kadar erken ve katı konursa o kadar çok ölçek, o kadar az keşif elde edilir. TDD, bu genel kalıbın yazılım alanına özgü, mekanikleştirilmiş ve geri bildirim gecikmesi asgariye indirilmiş bir uygulamasından ibarettir.